

Royal King Water Supply — Operations Management System

A two-portal web application that runs the daily operations of a packaged-water (can) delivery business: drivers log deliveries and expenses from the field, and the admin oversees customers, payments, fleet, and analytics from a single dashboard.

Prepared 14 July 2026 Repository: `royalkingadmin` Product app: `apps/web`

OVERVIEW

What the application is

Royal King Water Supply delivers 20-litre water cans to homes and offices on subscription. This system replaces paper registers with a mobile-friendly web app covering the full operational loop: **customer onboarding → daily delivery logging → payment tracking → expense capture → oversight and analytics**.

It is a single Next.js application serving two audiences through separate, role-gated portals. Drivers use it on their phones during routes; the admin uses it on desktop or mobile to supervise everything the drivers record and to manage the business's master data.

TYPE Operations web app	PORTALS Admin + Driver	FRAMEWORK Next.js 16 / React 19
DATABASE MongoDB (Mongoose)	DATA MODELS 7 collections	ROUTES 28 pages & APIs

USERS

Roles and what each one does

Admin BACK OFFICE

Business owner / manager. Signs in with credentials held in server environment variables.

- Dashboard with today's headline numbers (deliveries, drivers, vehicles, customers)
- Full delivery & cash-credit ledger — filter, edit, delete, export as PNG/PDF
- Payment tracking per delivery: Cash, UPI, or Not Paid
- Customer management (CRUD) with per-customer **history pages**: lifetime deliveries, cans, and amounts
- Driver and vehicle management, supply-point assignments
- Analytics charts filtered by date range, driver, or vehicle

Driver FIELD

Delivery staff. Signs in with a username/password account created by the admin; may have an assigned vehicle.

- Log a water delivery: pick customer (searchable), cans delivered / taken back, vehicle, payment status, notes
- Log a cash event: debit or fuel expense, with a compressed photo of the fuel bill
- Register a new customer on the doorstep
- Review their own recent logs (last 5 days by default, any older day via calendar)
- See and complete supply-point assignments

USE CASES

A day in the system

1. **Morning:** the admin reviews yesterday's ledger, marks amounts on deliveries that were logged without one, and checks Not-Paid entries. Supply-point assignments for the day are already scheduled (one-off or daily recurring).
2. **On route:** each driver logs deliveries as they happen — two taps for a routine subscription drop. The amount is auto-calculated from the customer's per-can rate. Empty cans taken back are recorded in the same entry.
3. **Expenses:** when a driver fuels the tanker or takes cash, they log it under Cash Credits; fuel entries require a bill photo, which is compressed on the phone and stored in Cloundinary.
4. **New customers:** a driver can register a customer at the door (name, phone, address, home/office, rate). The customer immediately becomes selectable for delivery logging.
5. **Evening / any time:** the admin audits the day per driver or vehicle, opens any customer's history page to answer "how many cans this month, how much is owed", and exports the filtered ledger as a PNG or PDF to share.

TECHNOLOGY

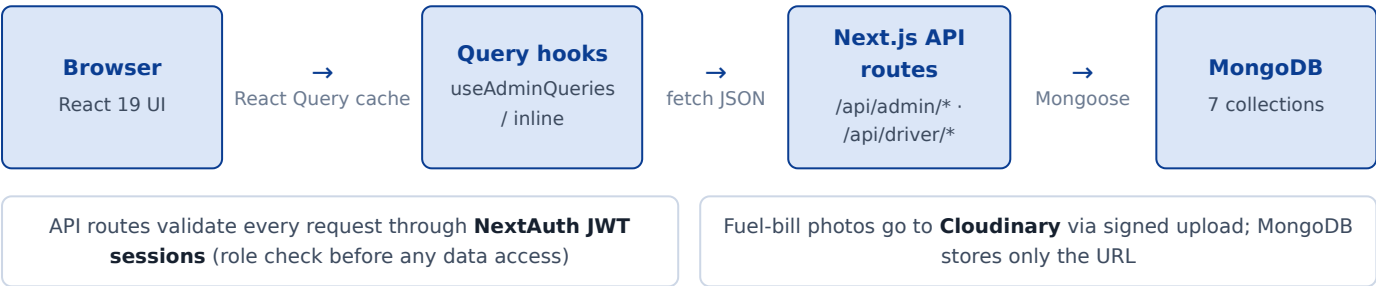
Stack

LAYER	CHOICE	NOTES
Framework	Next.js 16.1.5 (App Router), React 19	Pages and API routes in one deployable app
Language	TypeScript 5.9	Strict typing across pages, hooks, and API routes
Database	MongoDB via Mongoose 9.2	Cached connection; compound indexes on log queries

Auth	NextAuth v4, credentials + JWT sessions	Admin from env vars; drivers from DB with bcrypt hashes
Client data	TanStack React Query v5	Caching, pagination with <code>keepPreviousData</code> , background refetch
Media	Cloudinary	Fuel-bill images; signed uploads via raw HTTP (no SDK)
Styling	Plain CSS design system (<code>globals.css</code>)	CSS variables + utility classes; no Tailwind
Monorepo	Turborepo + npm workspaces	<code>apps/web</code> is the product; shared configs in <code>packages/</code>

ARCHITECTURE

How data flows



Every API route first validates the session and role, then queries MongoDB through Mongoose. The browser never talks to the database directly. React Query keeps a client-side cache keyed by filters and page number, so revisits and pagination steps render instantly from cache while fresh data loads in the background. Writes invalidate the relevant cache prefixes without blocking the UI.

List endpoints do the heavy lifting server-side: pagination, date-window math, and summary statistics are computed in MongoDB (aggregation bounded to the visible window), so the client only ever receives and renders the rows it shows.

DATA

Data model

COLLECTION	PURPOSE	KEY FIELDS
User	Driver accounts	name, username, bcrypt password, role, isActive, assignedVehicle → Vehicle
Customer	Subscribers	name, phone (unique), address, area, home/office/both, subscriptionCans, cashPerCan, securityDeposit, registeredDate, isActive, soft-delete flag
Vehicle	Tanker fleet	name, vehicleNumber (unique), capacity, isActive
SupplyLog	The ledger — one row per event	logType (water / cash), driver, customer, vehicle, suppliedAt, cansDelivered, cansTakenBack, amount, paymentStatus (cash / upi / not_paid), cashType (debit / fuel), bill image, notes, adminRemark
WaterSupplyPoint	Bulk supply locations	name, address, tankerTypes, isActive

Assignment	Scheduled point deliveries	supplyPoint, driver, tankerType, once/daily, status, completedAt
Entry	Legacy collection ledger	date, customerName, amountCollected, paymentMode, dieselExpense

SupplyLog is the heart of the system. Water deliveries and cash events share one collection, discriminated by `logType`, which lets a single indexed timeline serve the driver's log view, the admin ledger, per-customer history, and analytics. Indexes cover the hot paths: by date, by driver + date, by customer + date, and by logType + date.

INTERFACE

Screens

Admin portal

ROUTE	SCREEN
<code>/admin</code>	Dashboard — today's stats and recent deliveries
<code>/admin/supplies</code>	Deliveries & Cash Credits ledger — tabs, filters (date, month, driver, vehicle, payment), day-grouped tables with per-day serial numbers, add/edit/delete, PNG & PDF export
<code>/admin/customers</code>	Customer list — search, area filter, add/edit/delete; whole row opens the customer
<code>/admin/customers/[id]</code>	Customer history — profile, lifetime totals, paginated delivery log with month & payment filters
<code>/admin/amounts</code>	Analytics — delivery & registration charts by range, driver, vehicle
<code>/admin/drivers</code> · <code>/admin/vehicles</code>	Fleet and staff management
<code>/admin/assignments</code> · <code>/admin/supply-points</code>	Supply-point scheduling

Driver portal

One mobile-first page at `/driver` with three tabs — **Supply** (log delivery + recent list), **Cash** (log expense with bill photo + recent list), and **Register** (new customer). Log lists show the last 5 days grouped by day; a calendar filter fetches any older day on demand.

ENGINEERING

Performance design

The app is used on mid-range phones over mobile data, so recent work focused on keeping both reads and writes light:

- **Windowed pagination.** Unfiltered log views load only the last 5 days; Next/Prev walks older records 50 at a time. Summary statistics are aggregated over the same window, never the whole collection.

- **One query at a time.** On tabbed screens only the visible tab's query is active, halving load on open and after every save.
- **Non-blocking saves.** Creating a delivery, customer, or cash entry returns as soon as the write lands; list refreshes happen in the background while existing rows stay on screen.
- **Light rendering.** Offscreen day-groups skip layout and paint via `content-visibility: auto;` images load as Cloudinary thumbnails.
- **Index-backed queries.** Every list and history query is covered by a compound index; write endpoints avoid redundant round-trips by populating created documents in place.

SECURITY

Access control

- Every API route checks the JWT session and role before touching data — admin routes reject drivers and vice versa; drivers can only read and write their own logs.
- Driver passwords are bcrypt-hashed; the admin credential lives only in server environment variables.
- Inputs are validated server-side: ObjectId checks on all references, date/number validation, image type and 8 MB size limits on uploads.
- Customer deletion is a soft delete — history is preserved and remains viewable from the ledger.

WORTH CONSIDERING NEXT

There is a single shared admin account, no rate limiting on the login endpoint, and no audit trail of admin edits. None block current use, but all three are natural hardening steps as the team grows.